

Mahmoud Abdalla — Sprint 1 AI Usage Log

Sprint 1 — AI Usage Log

Team Member: Mahmoud Abdalla **Tool Used:** Claude (Claude Code CLI), model Claude Sonnet 5 **Note on chat links:** This work was done in the Claude Code CLI, which has no shareable web chat link. Full prompts/responses are summarized below; ask me to attach the raw transcript if the TA wants it verbatim.

Entry 1: Tech Stack Selection

- **Purpose of AI Use:** Planning / architecture decision
 - **Prompt/Response:** Asked Claude which stack to use for CareerConnect given it needs auth, a database, file upload, and an AI feature later, for a solo/small-team course project. Claude proposed two options (Next.js single-codebase vs. separate React+Express) and recommended Next.js + TypeScript + Prisma/SQLite + NextAuth + Tailwind for setup simplicity.
 - **AI-Suggested Content:** Full stack recommendation with rationale (single codebase, zero-config local DB, easy deploy).
 - **Validation:** Compared against project needs (registration/login, resume upload, future job postings/AI feature) and course scope (Sprint 1 only needs 2 working demo features). Confirmed Next.js's API routes cover both backend needs listed in the brief.
 - **Decision:** Accepted.
 - **Reflection:** Saved time versus researching stacks manually. Would have chosen something similar myself, but the recommendation was faster and included setup tradeoffs I hadn't considered (e.g. Prisma's relative-path SQLite gotcha, which did bite us later — see Entry 3).
 - **Responsible Person:** Mahmoud Abdalla
-

Entry 2: Project Scaffolding

- **Purpose of AI Use:** Implementation (setup/boilerplate)
 - **Prompt/Response:** Asked Claude to scaffold the Next.js app into the existing CareerConnect repo without disturbing the README/git history, then add Prisma, NextAuth, and bcrypt.
 - **AI-Suggested Content:** Ran `create-next-app`, merged generated files into the repo, initialized Prisma with a SQLite datasource, installed `next-auth`, `bcryptjs`, and related dev dependencies.
 - **Validation:** Ran `npm run build` and `npx tsc --noEmit`; fixed two real type errors it caught (a bad type cast in `auth.ts`, an incorrect Prisma Client import path) before moving on.
 - **Decision:** Accepted, with minor fixes applied.
 - **Reflection:** Useful for avoiding manual boilerplate typos. The generated `AGENTS.md`/`CLAUDE.md` files were auto-created by `next dev` itself (a Next.js 16 feature, not something Claude invented) — kept them since Next.js regenerates them anyway.
 - **Responsible Person:** Mahmoud Abdalla
-

Entry 3: Registration/Login + Resume Upload/Profile Features

- **Purpose of AI Use:** Coding (implementation)

- **Prompt/Response:** Asked Claude to build the two Sprint 1 demo features: credentials-based registration/login, and profile editing with resume upload.
 - **AI-Suggested Content:** NextAuth credentials provider with bcrypt password hashing, `/api/register`, `/api/profile` (GET/PATCH), `/api/profile/resume` (file upload with type/size validation), and the corresponding React pages (`/register`, `/login`, `/profile`).
 - **Validation:** Tested the full flow live in a browser (via the Claude in Chrome tool): registered a test account, logged in, edited the profile headline, uploaded a test PDF resume, confirmed the file and DB record persisted, and signed out — all verified working, not just “looks right in the code.”
 - **Decision:** Accepted, with one bug fixed during validation (see Entry 4).
 - **Reflection:** Generating the code was fast, but the AI’s first pass had a real bug that only surfaced when actually run — reinforces that “AI wrote it” is not the same as “it works,” and live testing was necessary, not optional.
 - **Responsible Person:** Mahmoud Abdalla
-

Entry 4: Debugging — SQLite Database Path Resolution

- **Purpose of AI Use:** Debugging
 - **Prompt/Response:** After live-testing surfaced a 500 error on registration (“Unable to open the database file”), asked Claude to diagnose it.
 - **AI-Suggested Content:** Identified that the Prisma CLI resolves a relative `DATABASE_URL` (`file:./dev.db`) relative to `prisma/schema`.prisma’s directory, while the running Next.js app resolves it relative to the project root — so the CLI created the DB at `prisma/dev.db` but the app looked for it at the project root. Fixed by switching `DATABASE_URL` to an absolute path.
 - **Validation:** Restarted the dev server and re-ran the full register→login→profile→upload flow in-browser; confirmed it succeeded end to end with no errors.
 - **Decision:** Accepted.
 - **Reflection:** A subtle, non-obvious Prisma/SQLite footgun that would have taken real trial-and-error to find manually. Also hardened the client-side error handling afterward (a failed `res.json()` on an empty error body was throwing an unhandled exception) as defense-in-depth.
 - **Responsible Person:** Mahmoud Abdalla
-

Entry 5: Generate 10 User Stories (Required Sprint 1 Task)

- **Purpose of AI Use:** Requirements elicitation / brainstorming (this is the specific AI Log entry required by the assignment brief)
- **Prompt/Response:** “Generate 10 user stories for CareerConnect, a job search and application tracking platform for job seekers and recruiters, based on this feature list: registration/profile management, resume upload, job posting management, job search/ filtering, application submission, application status tracking, application history dashboard, notifications/reminders, saved jobs, and AI-assisted resume feedback/job matching.” Claude returned 10 stories (US-01 through US-10) each in “As a , *I want* so that ____” format with suggested task breakdowns, covering both job-seeker and recruiter perspectives. Full list recorded in `docs/user-stories.md`.
- **AI-Suggested Content:** 10 user stories + task breakdowns (see `docs/user-stories.md`, “AI-Generated User Stories” section).
- **Validation:** Cross-checked every story against the system features listed in the project brief — all 10 map directly to a listed feature, no invented functionality. Not yet reviewed in a team meeting (solo team member at time of writing) — flagged as still needing that pass before being finalized as GitHub Issues.
- **Decision:** Modified before use — kept the stories as-is but flagged them explicitly as unvalidated-by-team in `docs/user-stories.md`, and added a required minimum of 5 additional team-generated stories to reach the 15-story minimum, kept in a clearly separate section per the assignment’s instructions not to mix AI and team-authored content.

- **Reflection:** The AI-generated stories were reasonable but generic — they cover the obvious CRUD paths for each feature but don't include anything creative or team-specific (e.g. nothing about the AI resume-feedback UX itself, nothing about deadline reminders' actual behavior). This is exactly the gap the “Team-Generated User Stories” section is meant to fill.
 - **Responsible Person:** Mahmoud Abdalla
-

Entry 6: Sprint 1 Tracker Artifact

- **Purpose of AI Use:** Project planning / documentation
 - **Prompt/Response:** Asked Claude to build a detailed, numbered checklist of every Sprint 1 deliverable from the assignment brief, so progress could be tracked across sessions.
 - **AI-Suggested Content:** A 59-item checklist across 9 categories (GitHub Setup, README, Sprint Planning, User Stories, Team Process, AI Log, Meeting Minutes, Code Demo, Submission Package), published as an interactive web page with clickable status tracking.
 - **Validation:** Manually reviewed every line against the assignment brief's “Activities” table and rubric to confirm nothing was missed or miscategorized.
 - **Decision:** Accepted.
 - **Reflection:** Useful for keeping a solo/small team oriented across a multi-week sprint with many small deliverables; the numbering makes it easy to reference specific items (“blocked on B5”) in conversation.
 - **Responsible Person:** Mahmoud Abdalla
-

Entry 7: Sprint Planning, Team Process, and GitHub Issues Setup

- **Purpose of AI Use:** Project planning / documentation / GitHub setup
- **Prompt/Response:** Asked Claude to set up the GitHub Project board, create GitHub Issues for the 10 user stories with labels and task checklists, and draft the Sprint Planning (backlog, prioritization, risks, effort estimation, capacity) and Team Process (workflow, branching, PR/review rules, DoR/DoD) documents required by the brief.
- **AI-Suggested Content:** A 5-column project board linked to the repo; 10 GitHub Issues (4 closed as already-implemented, 6 left in Backlog) with user-story, priority:*, and area:* labels; docs/sprint-planning.md and docs/team-process.md.
- **Validation:** Reviewed the backlog/priority/risk content against the actual state of the repo (e.g. risks around SQLite and local file storage are real, verified constraints, not generic filler). Confirmed the real Sprint 1 deadline (Sept 28, 2026) with the team member and had it incorporated into the risk section and Appendix A.
- **Decision:** Accepted, with one item flagged incomplete: the team capacity estimate (10 hrs/week) is a placeholder pending the team member's real number.
- **Reflection:** AI was efficient at generating the scaffolding of process documents (branching convention, DoR/DoD, PR checklist) that are fairly standard practice — but the actual risk assessment needed grounding in this specific project's real technical decisions (SQLite, local file storage) rather than generic “risks might include...” text, which required explicit correction/direction.
- **Responsible Person:** Mahmoud Abdalla

Ongoing

Every AI-assisted activity for the remainder of Sprint 1 (further coding, debugging, documentation, planning) will be appended to this log before it is exported to PDF for submission (see AI Log tracker item F4).